Procedural distributions

From PZwiki

English • čeština • Deutsch • español • français • italiano • 日本語 • 한국어 • polski • português • português do Brasil • русский • ไทย • Türkçe • українська • Tiếng Việt • 中文（简体） • 中文（繁體）

Modding

Give your feedback on the Modding Wiki **here (https://docs.google.com/forms/d/e/1FAIpQLSdxBWwKUcNP_KbzmmWo1eNc2QkpWk5dTj4PMhuNLY9_dc4IEw/viewform?usp=header)**!

Category:Modding • Scripts • Lua (API) • Java • User Interface • Modeling • Animation • Texturing • Mapping • Rendering • Translation

This page has been updated to an *unstable* beta version (42.7.0).

For the Build 41 page, please see this archived version.

Procedural distribution explains how the loot system of the game works and how you can add your own loot to the procedural distribution.

How does loot generation works

Containers in a room generate with different possibilities of distribution lists. Loot in containers is generated whenever they get loaded in the game or can respawn loot if the proper settings in the sandbox options are set.

Each distribution has a set amount of rolls, and each item entry has a set chance which is neither a weight nor a percentage. Every distribution item entries will be rolled the amount of rolls the distribution list has with a random chance generated based on some criteria such as loot spawn chance and the zombie population.

When loot is being spawned in a container, the event OnFillContainer triggers, allowing you to modify items which are spawning in or add other items.

Distribution structure

Vanilla loot distribution is defined in the following file:

```
ProjectZomboid/
└── media/
    └── lua/
        └── server/
            └── Items/
                └── ProceduralDistributions.lua
```

Each distribution lists are added to the table `ProceduralDistributions.list` in the following form:

```lua
ProceduralDistributions.list = {
    distributionName = { -- the distribution category
        rolls = 3,
        items = { -- can be empty
            "Item1", -- item n°1 in the distribution category
            10, -- the item n°1 roll chance
            "Item2", -- item n°2 in the distribution category
            5, -- the item n°2 roll chance
            ...
            "ItemN", -- item n°N in the distribution category
            3 -- the item n°N roll chance
        },
        junk = { -- ignores zombie density and has a x1.4 chance multiplier
            rolls = 1, -- number of rolls for junk
            items = { -- can be empty
                -- same way to list as items above
            }
        }
```

```
    },
    otherDistributionName = {
        ...
    },
    ...
}
```

As a real example, take for example:

```
ProceduralDistributions.list = {
    ...
    ComicStoreShelfComics = {
        isShop = true,
        rolls = 4,
        items = {
            "ComicBook_Retail", 50,
            "ComicBook_Retail", 20,
            "ComicBook_Retail", 20,
            "ComicBook_Retail", 10,
            "ComicBook_Retail", 10,
        },
        junk = {
            rolls = 1,
            items = {
                "ComicBook_Retail", 10,
            }
        }
    },
    ...
}
```

The chance value is **not** a weight **nor** a percentage! To chose your chance values, base yourself on existing vanilla distribution and the chance values of vanilla items!

Tags for distribution

Alongside the `rolls` tag for the `distributionName` and `junk` tables, you can add arguments which will modify how items spawn in and their conditions of spawn:

- `ignoreZombieDensity` - ignores the zombie density impact on item spawn chance.
- `isShop` - when not set to true:
  - Can be a stash. (Related to `stashChance`)
  - DrainableComboItem get random amount of uses.
  - HandWeapon can have lower condition (40% chance).
  - Items with head condition get reduced head condition (40% chance).
  - Items with sharpness condition get reduced sharpness condition (40% chance).
  - Bags get items inside them.
- `stashChance` - chance for the container to be a stash.
- `canBurn` - Food can be burnt (25% chance) or cooked.
- `isWorn` and `isTrash` - items spawn with lower condition, delta etc:
  - HandWeapon get reduced item condition.
  - Items with head condition get reduced head condition.
  - Items with sharpness condition get reduced sharpness condition.
  - DrainableComboItem get reduced item uses (i.e. batteries).
  - Impact on non-canned and edible (non can't eat) food:
    - Non-vermin, cookable and non-replacable on cooked will be either cooked or burnt (50% chance).
    - Non-rotten items will be rotten (75% chance) or have increased age (less fresh).
    - Have reduced food value.
- `isWorn` specifically:
  - Clothing items will have reduced condition, can be dirty (25% chance), bloody (1% chance) and/or have holes (25% chance).
- `isTrash` specifically:
  - Clothing items will have reduced condition, can be wet (25% chance), dirty (95% chance), bloody (10% chance) and/or have holes (75% chance).
- `isRotten` - non-rotten items will be rotten (75% chance) or have increased age (less fresh).
- `bags` - unsure, could be adding bags to the container.
- `maxMap` (integer) - limits the same item to a max amount. (UNSURE)

- `onlyOne` **DEPRECATED** - a tag which can be found in distributions but looks deprecated from the java.

How to add your own items to distributions

The main challenges you will face is properly understanding how loot generation actually works (see #How does loot generation works), finding the right distribution list and organizing the file which adds these. Inserting inside the procedural distribution needs to be done in `media/lua/server`.

There are many ways of achieving this, with the most common one, but worst one, being:

```
table.insert(ProceduralDistributions.list["DistributionListName"].items, "YourItem") -- add your item to the list
table.insert(ProceduralDistributions.list["DistributionListName"].items, 0.5) -- following its chance value
```

Items that are defined with a different module than `Base` need to have their module in the item name. For example, if you have an item named `MyItem` in the `MyMod` module, you need to add it as `MyMod.MyItem`.

A better way to handle this is to have every distributions and their items and chances in a singular table which is parsed through thanks to a loop and added with a single `table.insert` line.

Take this example with a few items and distribution lists:

```
local myDistribution = {
    GigamartPots = {
        items = {
            "GlassWine", 6, -- item, chance,
            "Fork", 10,
            "Mugl", 10,
            "Whetstone", 10,
            "Teacup", 10,
            "CDplayer", 10,
        },
        junk = { -- this one can be removed if you don't put anything in it
```

```lua
            "Mov_CoffeeMaker", 4,
            "HandTorch", 8,
        },
    },
    LibraryMilitaryHistory = {
        items = {
            "Book_Music", 20,
            "Book_Music", 10,
            "Doodle", 0.001,
        },
    },
    MechanicShelfElectric = {
        items = {
            "Battery", 10,
            "BatteryBox", 10,
            "Brochure", 2,
            "Broom", 10,
            "Bucket", 10,
        },
    },
}

-- caching for performance reasons
local ProceduralDistributions_list = ProceduralDistributions.list
local table_insert = table.insert

local function insertInDistribution(distrib)
    -- iterate through every given distributions
    for k,v in pairs(distrib) do
        -- cache this distribution list
        local ProceduralDistributions_list_k = ProceduralDistributions_list[k]

        -- insert items
        local items = v.items
        local ProceduralDistributions_list_k_items = ProceduralDistributions_list_k.items
        if items then
```

```lua
            for i = 1,#items do
                table_insert(ProceduralDistributions_list_k_items,items[i])
            end
        end

        -- insert junk
        local junk = v.junk
        local ProceduralDistributions_list_k_junk = ProceduralDistributions_list_k.junk
        if junk then
            for i = 1,#junk do
                table_insert(ProceduralDistributions_list_k_junk,junk[i])
            end
        end
    end
end

insertInDistribution(myDistribution)
```

Other table formating and functions can be used to insert in distribution.

Other loot tables

Some vanilla distribution tables will use generic items which are defined inside `ClutterTables`. Here's all the files that have clutter tables:

```
lua/
└── server/
    ├── Items/
    │   ├── Distribution_BinJunk.lua
    │   ├── Distribution_ClosetJunk.lua
    │   ├── Distribution_CounterJunk.lua
    │   ├── Distribution_DeskJunk.lua
    │   ├── Distribution_ShelfJunk.lua
    │   └── Distribution_SideTableJunk.lua
    └── Vehicles/
```

```
        ├── VehicleDistribution_GloveBoxJunk.lua
        ├── VehicleDistribution_SeatJunk.lua
        └── VehicleDistribution_TrunkJunk.lua
```

Backpacks that spawn in the world also use loot distribution tables which can be found in `BagsAndContainers` in the following file:

```
lua/
└── server/
    └── Items/
        └── Distribution_BagsAndContainers.lua
```

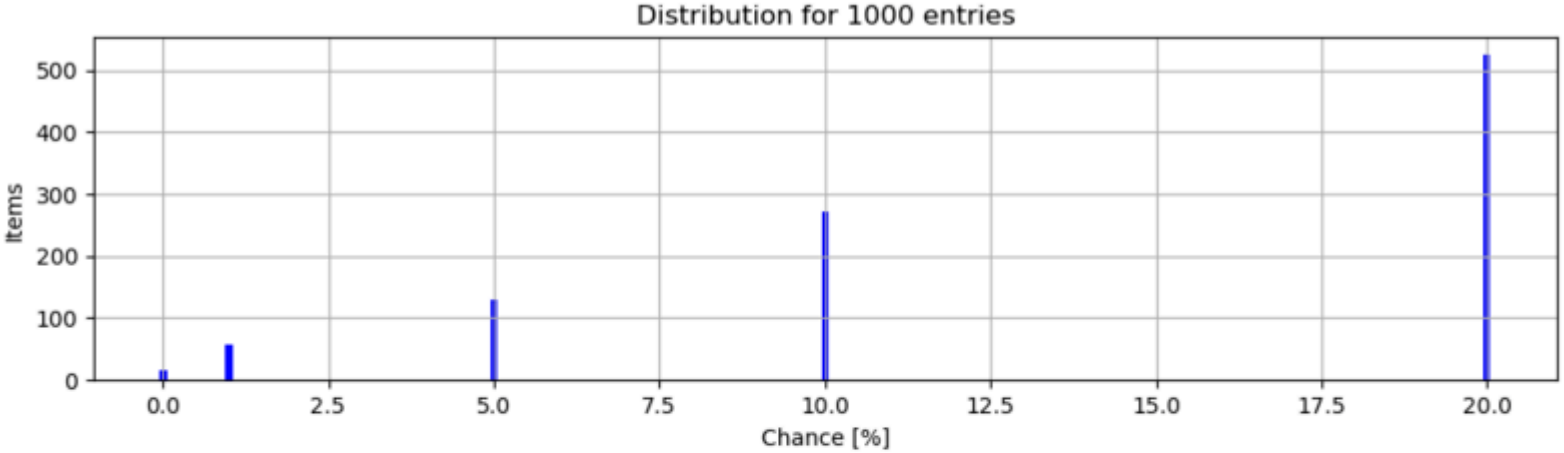
You can add items in these tables.

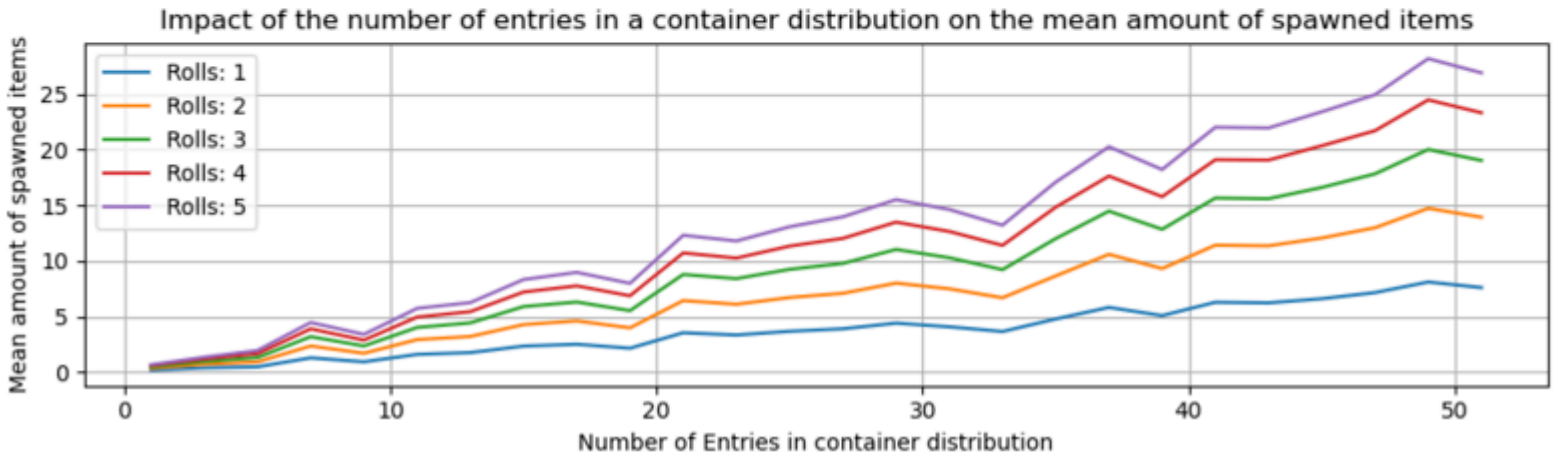
Accessing a container distribution in-game

Accessing a container distribution in-game can be achieved with LootZed which allows you to check for the various distribution lists for a container as well as listing each items and their chances of spawning in.

Problem with the system

Due to the way the game works, adding new items to a distribution list will ultimately increase the mean amount of items in this container. For a small demonstration of the issue, see the following image which is a simulation of a distribution list getting more and more entries by following a distribution of loot for different amount of rolls:





This is notably a major problem in some popular mods such as True Music (https://steamcommunity.com/sharedfiles/filedetails/?id=2613146550) addons, large gun mods or clothing packs which have modders add multiple items in a single distribution list and end up bloating the containers with loot. Other problems are that the game will ignore the container encumbrance limitations or there will be inequal amount of loot in different types of containers.

Solutions

Different methods exist to counter act having to add a lot of tiny items to containers. Some of the best methods can involve:

- having an item with variants which can change texture and data on the go. For example imagine a mod which adds game cards, it would require a hundred items but instead of creating an item script for each you can dynamically set the cards by utilizing functions that will change its texture, icon, name, tooltip etc on the go.

- creating dummy items. When the dummy item gets added to the container, you can intercept it with OnFillContainer, and replace it with another item which you can pick randomly in a list of items that are supposed to have similar spawn chances.

> Decreasing the overall chance of your items spawning is a band-aid solution which has its own problems. If your item is of a similar type as another one and should spawn with the same chances, this won't be possible since you decreased the chance of spawning it compared to the other item.

Distributions

Below is the full list of distributions as well as their different tags.

List of distributions and their different tags [Show]

See also

- PZTools - a tool to explore the loot distribution of the game.

External links

- Quick Guide: How To Mod The Loot Distribution System (Distributions.lua, ProceduralDistributions.lua) (https://theindiestone.com/forums/index.php?/topic/38165-quick-guide-how-to-mod-the-loot-distribution-system-distributionslua-proceduraldistributionslua/) — a 2021 guide to adding loot distribution.

Navigation

Modding

Retrieved from "https://pzwiki.net/w/index.php?title=Procedural_distributions&oldid=1310523"

Procedural distributions
From PZwiki

Share this page View history View source Discussion

More actions

More

Tools

- What links here alt ⇧ j
- Related changes alt ⇧ k
- Printable version alt ⇧ p
- Permanent link
- Page information
- Expand all